

System Design Fundamentals

For Java Full Stack, React & Cloud Developers

Load Balancing • Caching • Sharding • Replication • CAP Theorem • Microservices

Circuit Breakers • Rate Limiting • Kafka • Saga • CQRS • Event Sourcing

Observability • High Availability & Disaster Recovery

20 core system design topics, explained in simple terms with real examples, diagrams, and interview Q&A

Table of Contents

1. Introduction — Why System Design Matters for a Java Full Stack Developer
2. Load Balancer
3. API Gateway
4. Caching (Redis)
5. Database Sharding
6. Database Replication
7. CAP Theorem
8. Consistency Models
9. Horizontal vs Vertical Scaling
10. Microservices Architecture
11. Service Discovery (Eureka)
12. Circuit Breaker Pattern
13. Rate Limiting
14. Message Queues (Kafka / RabbitMQ)
15. Event-Driven Architecture
16. Distributed Transactions (Saga)
17. CQRS Pattern
18. Event Sourcing
19. Distributed Caching
20. Observability (Logs, Metrics, Traces)
21. High Availability & Disaster Recovery
22. Content Delivery Network (CDN)
23. Consistent Hashing
24. Database Indexing
25. Idempotency & Idempotent APIs
26. Health Checks, Readiness Probes & Blue-Green/Canary Deployments
27. Back-of-the-Envelope Capacity Estimation
28. Putting It All Together — A Complete System Design Example
29. Interview Questions & Real-Time Scenarios Across All Topics
30. Common System Design Issues & Real Fixes
31. Final Summary & Cheat Sheet

1. Introduction — Why System Design Matters for a Java Full Stack Developer

1.1 What "System Design" Actually Means

System design is the skill of deciding HOW the pieces of a real application fit together at scale — not writing a single class, but deciding: should this be one service or many? Where does data live, and how many copies? What happens when a server crashes? How do we handle 10x more traffic tomorrow? These 20 topics are the standard vocabulary every system design interview and real production architecture decision draws from.

1.2 How This Guide Is Organized

Each of the 20 topics (Sections 2-21) follows the same structure: a simple-terms explanation, WHY it's needed (the real problem it solves), a real diagram, how it applies specifically to a Java/Spring Boot + React + Cloud stack, a concrete example, real-time scenarios, and interview Q&A — exactly matching how these topics are actually asked about in real interviews and needed in real architecture discussions.

1.3 The Running Example Used Throughout

To keep every topic concrete, this guide uses one consistent running example: **"ShopNow"** — an e-commerce platform with a React frontend, Spring Boot microservices (Order, Payment, Inventory, Notification), running on Kubernetes in Azure — exactly the kind of system these 20 topics combine to build.

This guide deliberately keeps each topic focused and example-driven for interview readiness; for even deeper implementation detail on several of these topics (Microservices, Kafka, Redis, Observability, Kubernetes), see the other guides in this same documentation set, cross-referenced throughout.

2. Load Balancer

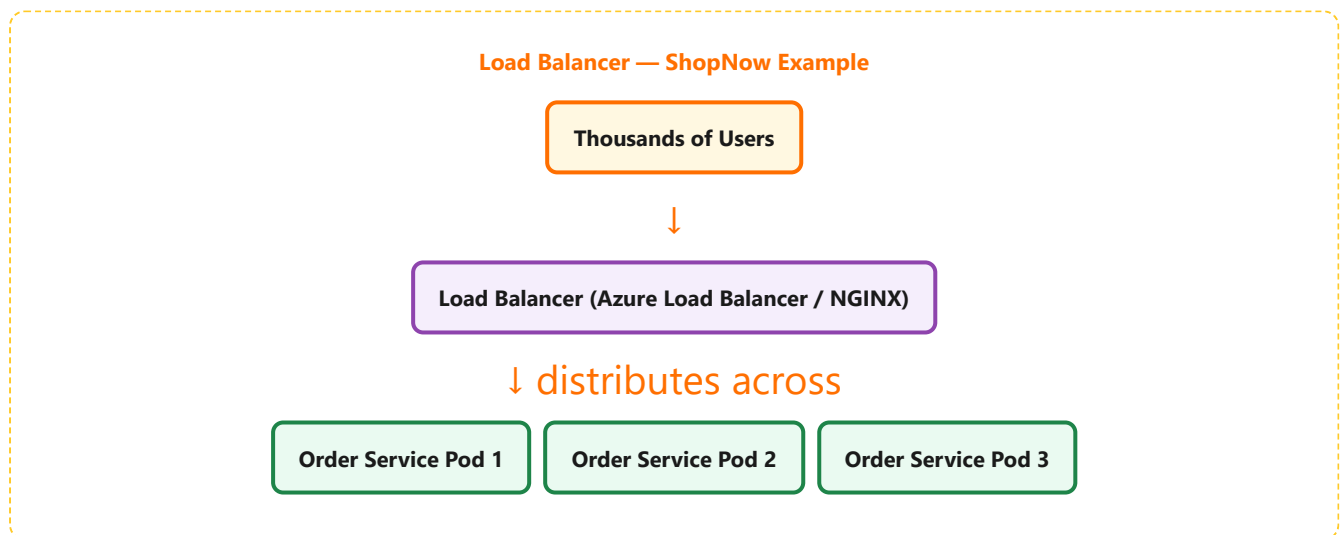
2.1 Simple Explanation

A Load Balancer sits in front of multiple identical server instances and decides which one handles each incoming request — like a traffic cop directing cars into whichever lane is least congested, instead of every car funneling into one lane.

2.2 Why It's Needed

Without a load balancer, all traffic hits ONE server — that server becomes both a performance bottleneck (can't handle more traffic than one machine allows) and a single point of failure (if it crashes, the entire application goes down).

2.3 Real Diagram



2.4 Java/Spring Boot + React + Cloud Application

Layer	Real Implementation
Cloud infrastructure	Azure Load Balancer (Layer 4) or Application Gateway (Layer 7) in front of AKS pods
Kubernetes-native	A Kubernetes Service (type: LoadBalancer or ClusterIP behind an Ingress) automatically load-balances across pod replicas
Spring Boot side	Every pod runs the identical Spring Boot JAR — completely stateless, so ANY instance can handle ANY request (critical requirement, Section 9)
React side	The frontend never knows or cares which specific backend instance served its request — it just calls one stable URL

2.5 Load Balancing Algorithms

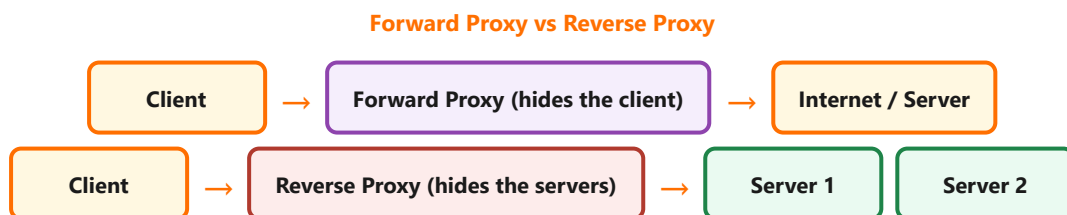
Algorithm	How It Works	Real Use
Round Robin	Cycles through instances in order	Simple default when all instances are equally capable
Least Connections	Sends traffic to the instance with the fewest active connections	Better when request processing times vary significantly
Weighted	Some instances get more traffic than others based on assigned weight	Gradually shifting traffic to a new, more powerful instance type
IP Hash / Sticky Sessions	The same client always routes to the same instance	Legacy apps needing session affinity (generally avoided in modern stateless design)

2.6 Real-Time Scenario

Real scenario: "During ShopNow's Black Friday sale, traffic increased 20x. Because Order Service pods are stateless and sit behind a Load Balancer + Horizontal Pod Autoscaler, Kubernetes automatically scaled from 4 pods to 40, and the Load Balancer immediately started distributing traffic across all of them — no code change, no manual intervention, the architecture simply absorbed the spike."

2.7 Related Concept: Forward Proxy vs Reverse Proxy

A **Forward Proxy** sits in front of CLIENTS, hiding who the client is from the server (e.g., a corporate proxy that all employee traffic goes through before reaching the internet). A **Reverse Proxy** sits in front of SERVERS, hiding how many/which servers exist from the client — a Load Balancer is actually a specific type of reverse proxy, and tools like NGINX or an API Gateway (Section 3) are also reverse proxies with extra capabilities layered on top (SSL termination, caching, routing).



Real scenario: "ShopNow's NGINX layer in front of its Kubernetes Ingress is a reverse proxy — it terminates SSL/TLS once at the edge (so individual Spring Boot pods don't each need to manage certificates) and then forwards plain HTTP internally to whichever healthy pod the Load Balancer selects."

2.8 Interview Q&A

Q1. What's the difference between Layer 4 and Layer 7 load balancing?

Layer 4 balances based on IP/port only (fast, protocol-agnostic); Layer 7 balances based on full HTTP details (URL path, headers), enabling smart routing like sending /orders/* to one service and /payments/* to another.

Q2. Why must backend services be stateless for load balancing to work well?

If a service stores session state in its own local memory, a client routed to a DIFFERENT instance on their next request won't find that state — stateless services (with any needed state externalized to Redis/a database) let ANY instance handle ANY request interchangeably.

3. API Gateway

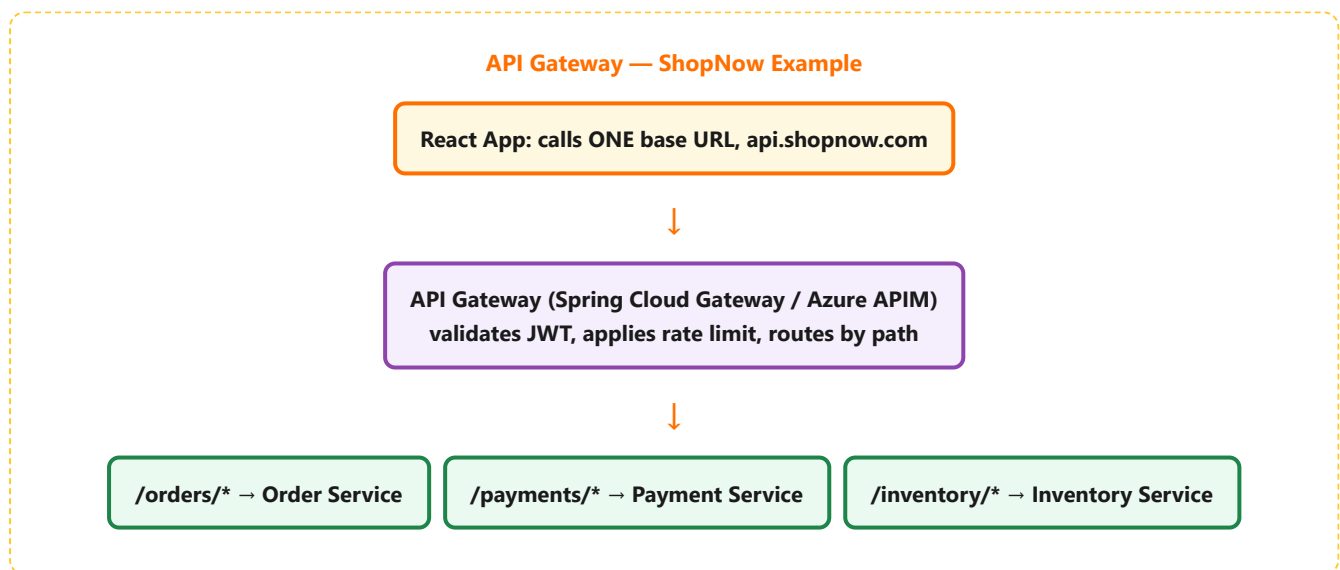
3.1 Simple Explanation

An API Gateway is a single front door that all client requests pass through before reaching any backend microservice — it handles routing, authentication, and rate limiting centrally, so individual microservices don't each have to implement these concerns themselves.

3.2 Why It's Needed

Without a gateway, the React frontend would need to know the address of EVERY microservice individually, implement authentication logic repeatedly, and every service would need its own rate limiting — a maintenance nightmare as the number of services grows.

3.3 Real Diagram



3.4 Java/Spring Boot + React + Cloud Application

```
// Real Spring Cloud Gateway route configuration
spring:
  cloud:
    gateway:
      routes:
        - id: order-route
          uri: lb://ORDER-SERVICE
          predicates: [ Path=/orders/** ]
        - id: payment-route
          uri: lb://PAYMENT-SERVICE
          predicates: [ Path=/payments/** ]
```

Full real implementation detail (JWT validation, rate limiting policies) is covered in the Microservices guide, Section 12, and the API Development guide, Section 21. React always talks to this ONE gateway URL — it never needs to know that "Order Service" and "Payment Service" are even separate deployments.

3.5 Real-Time Scenario

Real scenario: "ShopNow onboarded a new partner needing API access — instead of modifying every single microservice, the team configured ONE new rate-limiting policy in the API Gateway scoped to that partner's API key, live in minutes, with zero code changes to any backend service."

3.6 Interview Q&A

Q1. What's the difference between an API Gateway and a Load Balancer?

A Load Balancer distributes traffic across identical instances of ONE service. An API Gateway routes traffic to DIFFERENT services based on the request, plus handles cross-cutting concerns (auth, rate limiting) — many real Gateways use a Load Balancer internally for each service they route to.

Q2. What happens if the API Gateway itself goes down?

It becomes a single point of failure for the whole system — real production Gateways run as multiple replicas behind their own load balancer, exactly like any other stateless service, to avoid this risk.

4. Caching (Redis)

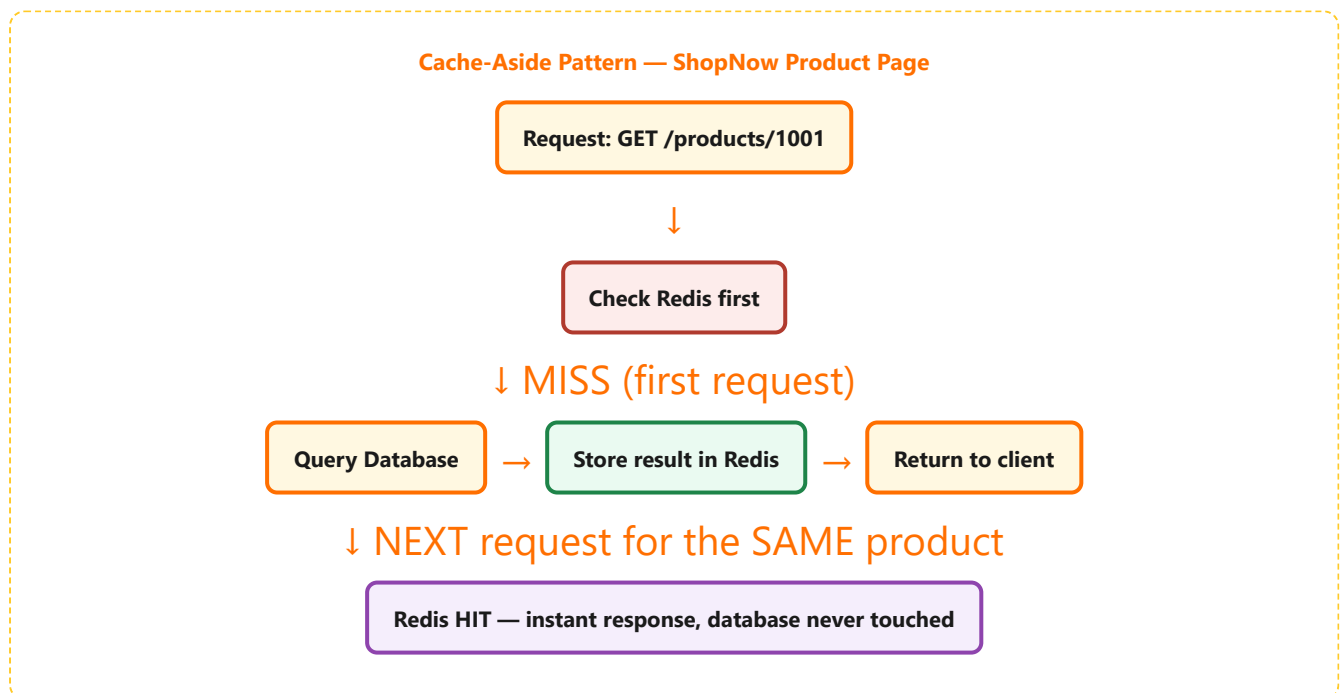
4.1 Simple Explanation

Caching stores a copy of frequently-needed data somewhere much faster to access than its original source (usually a database) — like keeping today's most-used files on your desk instead of walking to the archive room every single time.

4.2 Why It's Needed

Every database query costs time and database resources — if the same data (e.g., a product catalog page) is requested thousands of times per minute, re-querying the database every single time wastes resources and adds unnecessary latency.

4.3 Real Diagram



4.4 Java/Spring Boot + React + Cloud Application

```
@Cacheable(cacheNames = "products", key = "#id")
public ProductDto getProduct(Long id) {
    return mapper.toDto(productRepository.findById(id).orElseThrow());
}
```

Full production-grade Redis implementation (cache providers, TTL, eviction, the critical per-JVM-cache multi-instance gotcha) is covered in complete depth in the Spring & Spring Boot guide, Section 52-53, and the Web/Network/Redis guide, Sections 23-35. On the React side, caching also happens client-side via libraries like React Query, avoiding even the network round-trip for recently-fetched data.

4.5 Real-Time Scenario

Real scenario: "ShopNow's product catalog page load time dropped from 200ms to 4ms after adding Redis caching with a 10-minute TTL — since product details rarely change minute-to-minute, the small staleness window was a trivial trade-off for a 50x speed improvement and dramatically reduced database load."

4.6 Interview Q&A

Q1. What is cache invalidation, and why is it considered hard?

Knowing exactly WHEN cached data has become stale and must be refreshed — hard because any write path bypassing the normal cache-eviction logic (e.g., a direct database fix) leaves stale data being served until the TTL naturally expires.

Q2. Why must Redis (not a local in-memory cache) be used in a multi-instance microservices deployment?

A local per-JVM cache means each of the (e.g.) 10 running pod replicas has its OWN separate cache — an update on one pod is invisible to the other 9, causing inconsistent results depending on which pod serves a request; Redis provides one shared cache all instances see consistently.

5. Database Sharding

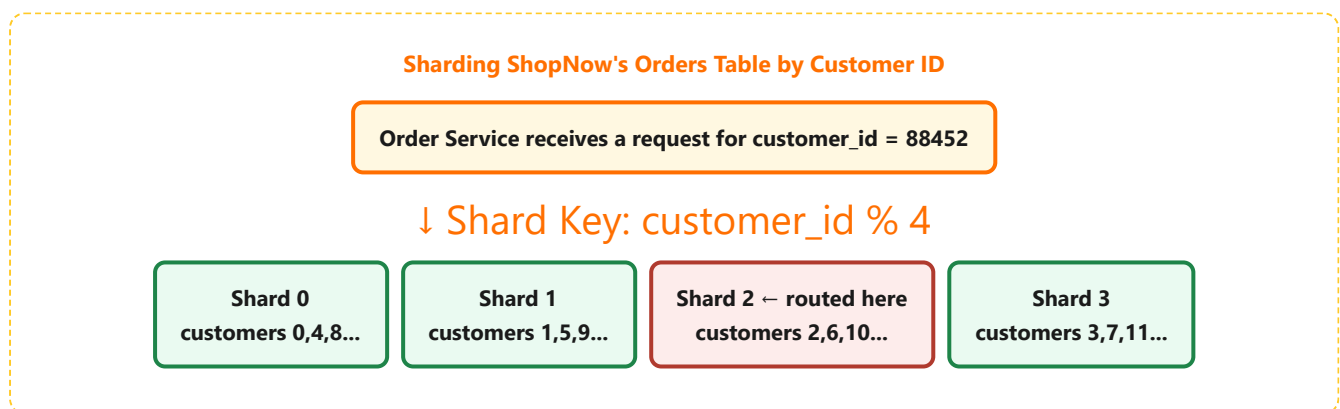
5.1 Simple Explanation

Sharding splits ONE large database's data across MULTIPLE separate database instances, each holding a SUBSET of the total rows — like splitting one giant phone book alphabetically across several smaller books (A-H in book 1, I-P in book 2) instead of one massive book everyone has to search through.

5.2 Why It's Needed

A single database server has physical limits — CPU, memory, disk I/O — once a table grows to hundreds of millions of rows, even well-indexed queries slow down, and vertical scaling (Section 8) alone eventually hits a hardware ceiling.

5.3 Real Diagram



5.4 Sharding Strategies

Strategy	How It Works	Real Trade-Off
Range-Based	Shard by a value range (e.g., customer IDs 1-1M in shard 1, 1M-2M in shard 2)	Simple, but can create "hot shards" if data isn't evenly distributed across ranges
Hash-Based	Shard key is hashed, result determines which shard (e.g., <code>customer_id % number_of_shards</code>)	Even distribution, but re-sharding (adding more shards) requires re-hashing and moving significant data
Directory-Based	A lookup table maps each key to its shard explicitly	Most flexible (easy to rebalance), but the lookup table itself becomes a critical dependency

5.5 Java/Spring Boot Application

Spring Boot applications typically implement sharding via a custom `AbstractRoutingDataSource` that picks the correct shard's `DataSource` based on the current request's shard key (e.g., customer ID), or by using a database/proxy layer (like Vitess for MySQL) that handles shard routing transparently below the application layer.

5.6 Real-Time Scenario

Real scenario: "ShopNow's single Orders database grew to 2 billion rows, and even indexed queries started taking seconds. Sharding by customer_id across 8 shards brought each shard down to a manageable ~250 million rows, restoring millisecond query times — the trade-off was that any query needing data across MULTIPLE customers (like a company-wide sales report) now requires querying all 8 shards and aggregating results in application code."

5.7 Interview Q&A

Q1. What's the biggest challenge sharding introduces?

Cross-shard queries/joins become much harder — a query needing data spanning multiple shards must query each shard separately and combine results in application code, since the database itself can no longer join across shard boundaries.

Q2. How do you choose a good shard key?

Pick a key that distributes data evenly (avoiding hot shards) and matches the MOST common query pattern (e.g., "get all orders for this customer" — sharding by customer_id keeps that query within one shard, avoiding expensive cross-shard fan-out).

6. Database Replication

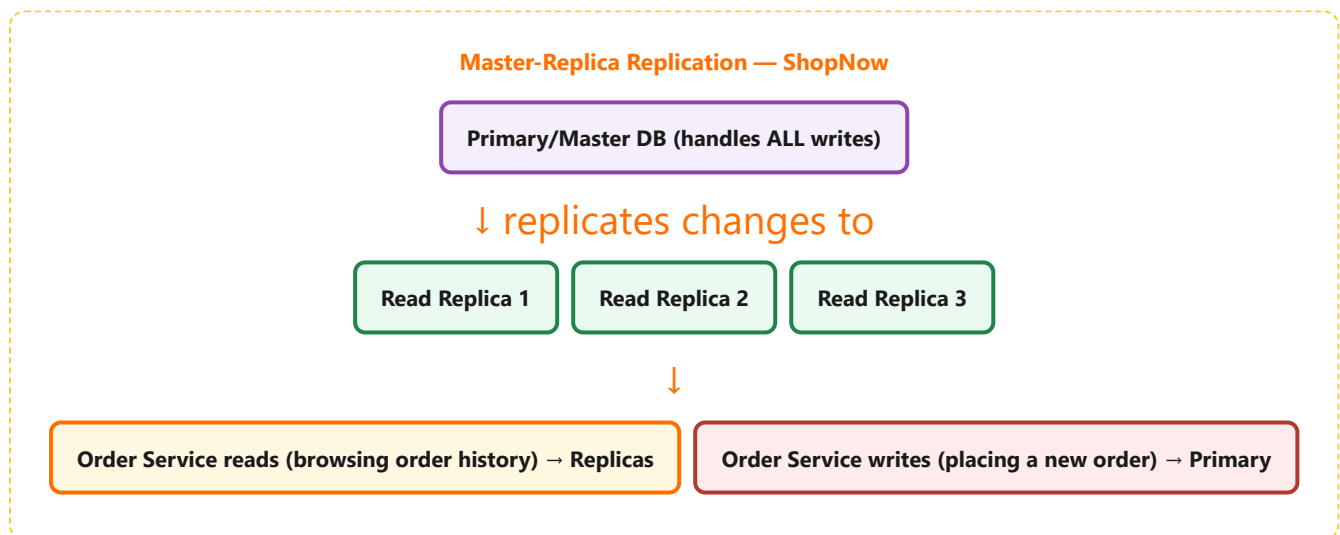
6.1 Simple Explanation

Replication keeps multiple COPIES of the same data on different database servers — unlike sharding (which SPLITS data across servers), replication DUPLICATES the same data for redundancy and read scalability.

6.2 Why It's Needed

A single database server is a single point of failure (if it crashes, the whole app loses data access) and a read bottleneck (every single read query hits the same one server). Replication solves both.

6.3 Real Diagram



6.4 Synchronous vs Asynchronous Replication

Type	How It Works	Trade-Off
Synchronous	The write isn't considered complete until it's confirmed on the replica(s) too	Strong consistency, but slower writes (waiting for network round trip to replicas)
Asynchronous	The write completes on the primary immediately; replicas catch up shortly after	Faster writes, but a brief window where replicas have stale data ("replication lag")

6.5 Java/Spring Boot Application

```
// Real Spring Boot read/write splitting using multiple DataSources
@Bean
@Primary
public DataSource writeDataSource() { return primaryDbConfig(); }

@Bean
public DataSource readDataSource() { return replicaDbConfig(); }

@Transactional(readOnly = true) // hints the routing layer to use the READ replica
public List<OrderDto> getOrderHistory(Long customerId) { ... }
```

6.6 Real-Time Scenario

Real scenario: "A customer places an order (written to the Primary), then IMMEDIATELY refreshes their order history page — if that read hits a replica with even 200ms of replication lag, the new order might briefly not appear, confusing the customer. ShopNow's fix: force reads immediately following a write for the SAME user to temporarily route to the Primary for a short window, then fall back to replicas afterward."

6.7 Interview Q&A

Q1. What is replication lag, and why does it matter?

The delay between a write completing on the primary and that change becoming visible on a replica — it matters because a read immediately following a write might hit a replica that hasn't caught up yet, returning stale data (a real consistency trade-off, connecting directly to the CAP Theorem, Section 7).

7. CAP Theorem

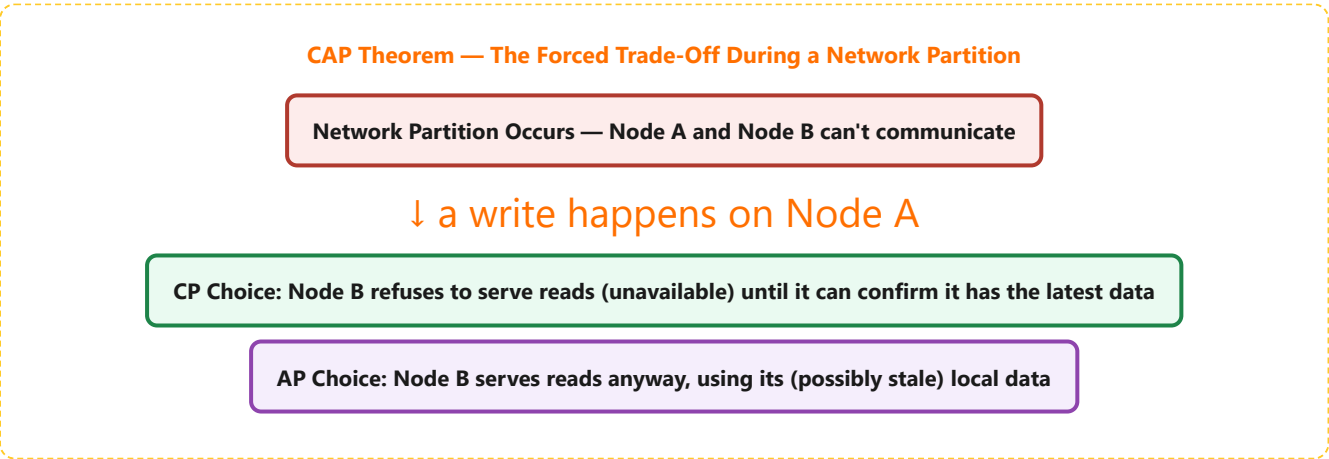
7.1 Simple Explanation

CAP Theorem states that a distributed database system can only guarantee TWO of these THREE properties at the same time: **Consistency** (every read sees the latest write), **Availability** (every request gets a response, even if not the latest data), and **Partition Tolerance** (the system keeps working even if network communication between nodes breaks).

7.2 Why It's Needed (as a Concept)

Network partitions (communication failures between servers) WILL happen in any real distributed system — CAP Theorem forces architects to consciously decide: when a partition occurs, do we sacrifice Consistency (keep serving requests with possibly-stale data) or Availability (refuse to respond rather than risk incorrect data)?

7.3 Real Diagram



7.4 Real Systems and Their CAP Choice

System	CAP Choice	Real Reasoning
Traditional RDBMS (single primary, e.g., banking core ledger)	CP	A bank absolutely cannot show an incorrect balance — consistency is non-negotiable, even if it means brief unavailability during a failover
Cassandra / DynamoDB	AP (tunable)	A product catalog can tolerate showing slightly stale data (an old price for a few seconds) rather than being unavailable entirely
MongoDB (default config)	CP-leaning	Prioritizes consistency by default, configurable toward AP for specific use cases

7.5 Real-Time Scenario

Real scenario: "ShopNow's Payment Service (backed by a strongly consistent relational database) chooses CP — during a network partition, it would rather briefly reject payment requests than risk double-charging or losing a transaction. ShopNow's Product Recommendation Service (backed by a NoSQL store) chooses AP — showing a slightly outdated recommendation is a far better trade-off than the recommendations widget breaking the entire page."

7.6 Interview Q&A

Q1. Does CAP Theorem mean you must choose CA (Consistency + Availability) and ignore Partition Tolerance?

No — in any REAL distributed system, network partitions are inevitable, so Partition Tolerance isn't really optional; the real-world choice is between CP and AP specifically WHEN a partition occurs, not a genuine three-way free choice.

Q2. How would you decide whether a specific microservice should favor CP or AP?

Ask: "What's worse — showing slightly stale data, or the service being unavailable?" Financial/inventory-critical services usually favor CP; content/recommendation/catalog-type services usually favor AP.

8. Consistency Models

8.1 Simple Explanation

Consistency models describe exactly WHAT GUARANTEE a system gives about seeing the latest data — ranging from "you always see the absolute latest write, instantly" (strong consistency) to "you'll eventually see it, just maybe not immediately" (eventual consistency).

8.2 The Main Consistency Models

Model	Guarantee	Real Example
Strong Consistency	Every read immediately reflects the most recent write, everywhere	A bank account balance — must always be exactly correct
Eventual Consistency	Given enough time with no new writes, all replicas WILL converge to the same value — but there's a window where they might differ	A social media "like count" — briefly showing slightly different counts to different users is acceptable
Read-Your-Own-Writes Consistency	A user always sees their OWN most recent write immediately, even if other users might briefly see stale data	You update your profile picture and see it change instantly, even if a friend viewing your profile sees the old one for a few seconds
Causal Consistency	Operations that are causally related (a reply to a comment) are seen in the correct order by everyone, though unrelated operations may not be	Chat applications — replies never appear before the message they're replying to

8.3 Real Diagram

Eventual Consistency — ShopNow Product Review Count

User submits a new review

↓ written to Node A immediately

Node A: review count = 101 (updated instantly)

↓ replicated asynchronously, takes a moment

Node B: review count = 100 (briefly stale) → eventually becomes 101

8.4 Real-Time Scenario

Real scenario: "ShopNow's review count uses eventual consistency (a NoSQL store) — a brief delay before a new review count appears to ALL users is completely acceptable. ShopNow's inventory count during checkout uses strong consistency

(via pessimistic locking in the relational Inventory database) — showing an item as available when it's actually sold out would create a genuinely bad customer experience and an operational headache."

8.5 Interview Q&A

Q1. Why would a system deliberately choose eventual consistency over strong consistency?

Strong consistency typically requires more coordination between nodes (higher latency, lower availability during partitions), while eventual consistency allows much higher availability and performance — the right choice depends entirely on whether the specific data can tolerate brief staleness.

9. Horizontal vs Vertical Scaling

9.1 Simple Explanation

Vertical Scaling means making ONE server bigger/more powerful (more CPU, more RAM). **Horizontal Scaling** means adding MORE servers running the same application, sharing the load between them — like hiring more cashiers (horizontal) versus giving one cashier superhuman speed (vertical).

9.2 Real Diagram

Vertical vs Horizontal Scaling — ShopNow Order Service

Vertical: 1 pod, upgraded from 2 CPU/4GB to 8 CPU/16GB

Horizontal: 4 pods, each still 2 CPU/4GB, load-balanced together

9.3 Full Comparison

Aspect	Vertical Scaling	Horizontal Scaling
Implementation	Resize the existing server/VM	Add more identical server/pod instances
Upper limit	Hard ceiling — eventually no bigger hardware exists	Practically limitless — keep adding more instances
Downtime to scale	Often requires a restart (resizing a VM)	None — new instances just join the pool
Fault tolerance	Still one server — a single point of failure	Naturally fault-tolerant — one instance failing doesn't take down the whole service
Application requirement	None — works with any application, even stateful ones	Requires the application to be stateless (Section 2.4) to work correctly
Cost model	Cost increases non-linearly at the high end (premium hardware)	Cost scales more linearly and predictably with commodity instances

9.4 Java/Spring Boot + Kubernetes Application

Kubernetes' Horizontal Pod Autoscaler (Docker & Kubernetes guide, Section 23) is horizontal scaling automated — adding more Spring Boot pod replicas as CPU/memory utilization rises. Vertical scaling in Kubernetes means increasing a pod's CPU/memory resource requests/limits — still possible, but real cloud-native architectures lean heavily toward horizontal scaling as the primary strategy.

9.5 Real-Time Scenario

Real scenario: "ShopNow initially tried vertical scaling — upgrading their single Order Service VM repeatedly during growth. They eventually hit the largest available VM size on Azure and STILL needed more capacity for Black Friday — this hard ceiling is exactly why they re-architected to run multiple horizontally-scaled Spring Boot pods behind a load balancer instead, removing any future capacity ceiling."

9.6 Interview Q&A

Q1. Why is horizontal scaling generally preferred in modern cloud-native architectures?

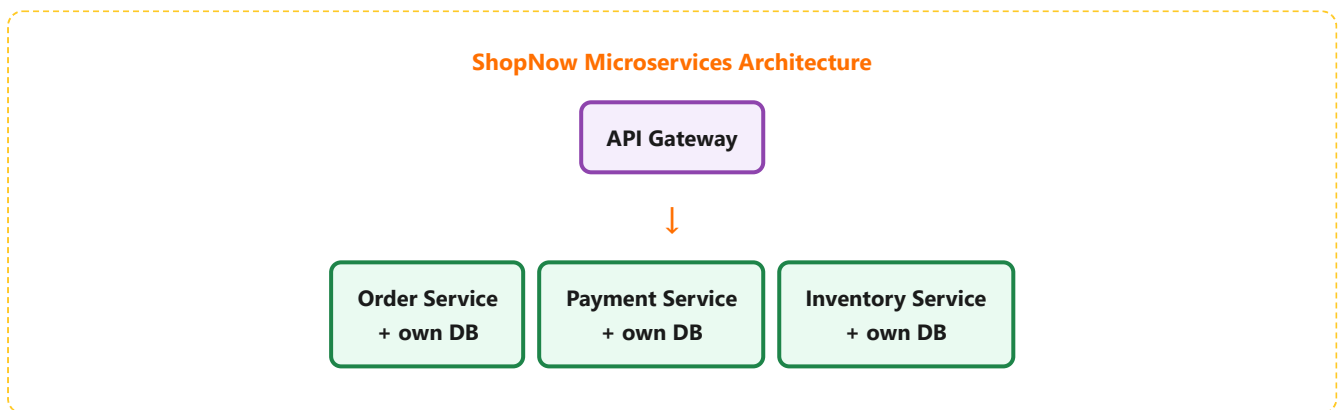
It has no hard ceiling, provides natural fault tolerance (one instance failing doesn't take down the service), and requires no downtime to add capacity — the trade-off is that it requires the application to be designed as stateless from the start.

10. Microservices Architecture

10.1 Simple Explanation

Microservices architecture splits one large application into many small, independently deployable services, each owning a specific business capability — instead of one giant ShopNow application, there's a separate Order Service, Payment Service, Inventory Service, each with its own codebase, database, and deployment lifecycle.

10.2 Real Diagram



10.3 Why It's Needed

Independent deployment (release Payment Service without touching Order Service), independent scaling (scale only Inventory Service during a flash sale), and team autonomy (different teams own different services without stepping on each other) — full complete depth already covered in the dedicated Microservices guide.

10.4 Monolith vs Microservices Quick Recap

Aspect	Monolith	Microservices
Deployment	One unit, all-or-nothing	Independent per service
Database	Usually shared	Database per service (Section 5's sharding, applied per-service)
Communication	In-process method calls	Network calls (REST/Kafka) — this entire guide's topics (Sections 11-13, 15) exist to handle the complexity this introduces

10.5 Real-Time Scenario

Real scenario: "ShopNow's Inventory Service needs to scale to 30 replicas during a flash sale, while Payment Service only ever needs 5 — in a monolith, you'd have to scale the ENTIRE application together, wasting resources; as separate microservices, each scales independently to exactly what it needs."

10.6 Interview Q&A

Q1. What's the single biggest trade-off microservices introduce compared to a monolith?

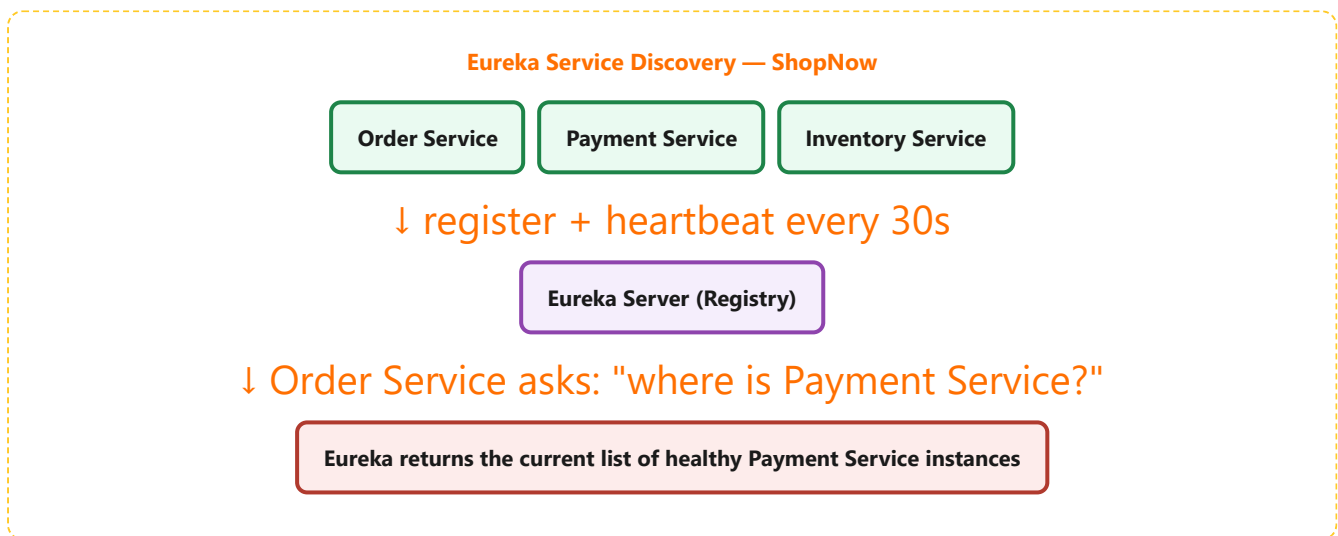
Distributed systems complexity — network latency, partial failures, and data consistency across services that used to be simple in-process concerns in a monolith become real engineering challenges requiring dedicated patterns (Circuit Breaker, Saga, Service Discovery — all covered in this guide).

11. Service Discovery (Eureka)

11.1 Simple Explanation

In a dynamic environment where service instances start, stop, and scale constantly (especially in Kubernetes), Service Discovery lets services find each other BY NAME rather than by hardcoded IP addresses that would break the moment an instance restarts with a new IP.

11.2 Real Diagram



11.3 Java/Spring Boot Application

```
@FeignClient(name = "payment-service") // "payment-service" resolved via Eureka automatically — no hardcoded IP anywhere
public interface PaymentFeignClient {
    @PostMapping("/payments") PaymentResponse process(@RequestBody PaymentRequest request);
}
```

Full real Eureka setup (server/client configuration, health check integration) is covered in complete depth in the Microservices guide, Section 11. Note that in a Kubernetes-native deployment, Kubernetes' own built-in DNS-based service discovery often replaces or complements Eureka, since Kubernetes already solves this exact problem at the platform level.

11.4 Real-Time Scenario

Real scenario: "When Payment Service scales from 2 to 8 pods during a sale, Eureka's registry reflects all 8 healthy instances within seconds — Order Service's Feign client automatically load-balances across all of them, with zero manual configuration change needed anywhere."

11.5 Interview Q&A

Q1. What happens if Eureka itself goes down?

Eureka clients cache the last known registry locally, so brief Eureka downtime doesn't immediately break existing service-to-service calls — but new service instances can't register until Eureka recovers, so Eureka itself is usually run as a small, highly available cluster.

12. Circuit Breaker Pattern

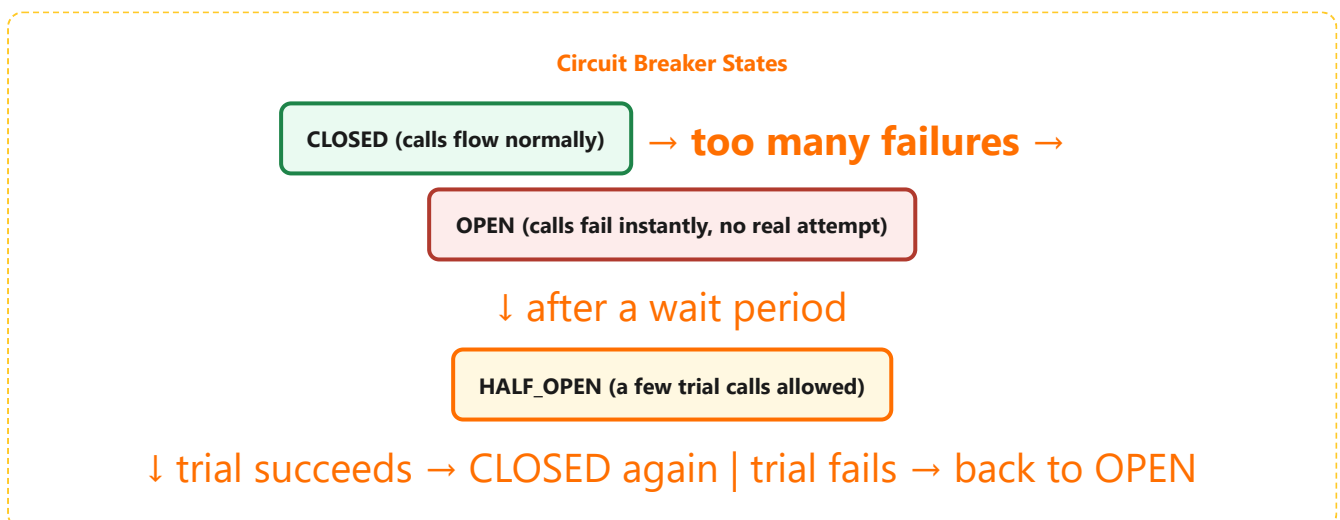
12.1 Simple Explanation

A Circuit Breaker monitors calls to a downstream service — if that service starts failing repeatedly, the breaker "opens" and immediately fails fast (without even attempting the call) for a period, giving the struggling service time to recover, instead of piling on more requests it can't handle.

12.2 Why It's Needed

Without a Circuit Breaker, if Payment Service becomes slow/unresponsive, Order Service keeps calling it and waiting — eventually Order Service's own threads get exhausted waiting, and IT becomes unresponsive too, cascading the failure across the whole system.

12.3 Real Diagram



12.4 Java/Spring Boot Application (Resilience4j)

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
public PaymentResponse processPayment(PaymentRequest request) {
    return paymentFeignClient.process(request);
}

public PaymentResponse paymentFallback(PaymentRequest request, Throwable t) {
    return PaymentResponse.pending("Payment queued for retry – service temporarily unavailable");
}
```

Full detail on Resilience4j configuration, combining Circuit Breaker with Retry/Bulkhead/Timeout, is covered in the Microservices guide, Section 18 and Section 40.

12.5 Real-Time Scenario

Real scenario: "When ShopNow's third-party payment gateway had a 10-minute outage, the Circuit Breaker opened after detecting the failure pattern — Order Service immediately started returning a graceful 'payment queued' fallback response instead of hanging on every checkout attempt, keeping the rest of the site fully responsive throughout the outage."

12.6 Interview Q&A

Q1. How is a Circuit Breaker different from just adding a timeout?

A timeout limits how long ONE call waits, but still makes every single call and waits for it to fail every time. A Circuit Breaker tracks the FAILURE RATE over time and, once a threshold is crossed, stops even attempting calls for a period — failing fast without wasting resources on calls likely to fail anyway.

13. Rate Limiting

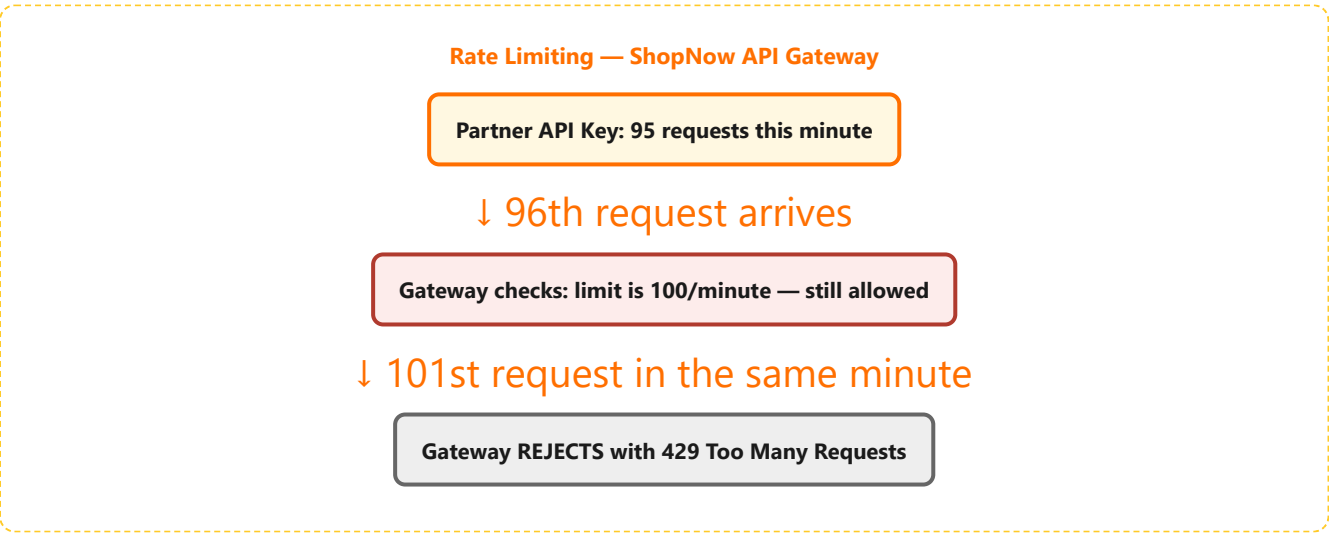
13.1 Simple Explanation

Rate Limiting caps how many requests a client (a user, an API key, an IP address) can make within a time window — like a "one ticket per person per hour" rule at a busy amusement park ride, preventing any single visitor from monopolizing capacity.

13.2 Why It's Needed

Without rate limiting, a buggy client stuck in a retry loop, a traffic spike, or a malicious/abusive client could overwhelm a service, degrading the experience for every OTHER legitimate user.

13.3 Real Diagram



13.4 Rate Limiting Algorithms

Algorithm	How It Works
Token Bucket	Tokens refill at a fixed rate; each request consumes a token, rejected once the bucket is empty
Sliding Window	Counts requests in a rolling time window, rejecting once the limit is exceeded
Fixed Window	Simple counter reset every fixed interval — can allow bursts right at window boundaries

13.5 Java/Spring Boot Application (Redis-Backed)

```
@Service
@RequiredArgsConstructor
public class RateLimiterService {
    private final RedisTemplate<String, String> redisTemplate;

    public boolean isAllowed(String apiKey, int maxPerMinute) {
        String key = "ratelimit:" + apiKey;
        Long count = redisTemplate.opsForValue().increment(key);
        if (count == 1) redisTemplate.expire(key, Duration.ofMinutes(1));
        return count <= maxPerMinute;
    }
}
```

13.6 Real-Time Scenario

Real scenario: "A ShopNow partner's integration had a bug causing it to retry every failed request instantly in a tight loop — without rate limiting, this would have overwhelmed the Order Service. The Gateway's rate limit rejected the excess requests with 429s, protecting the rest of the platform while the partner's team fixed their retry logic."

13.7 Interview Q&A

Q1. Where should rate limiting be enforced — the client, the Gateway, or each individual service?

Primarily at the API Gateway (centralized, consistent enforcement across all services, Section 3) — individual services MAY add their own additional limits for especially expensive operations, but relying on the client to self-limit is never sufficient since a malicious or buggy client won't respect it.

14. Message Queues (Kafka / RabbitMQ)

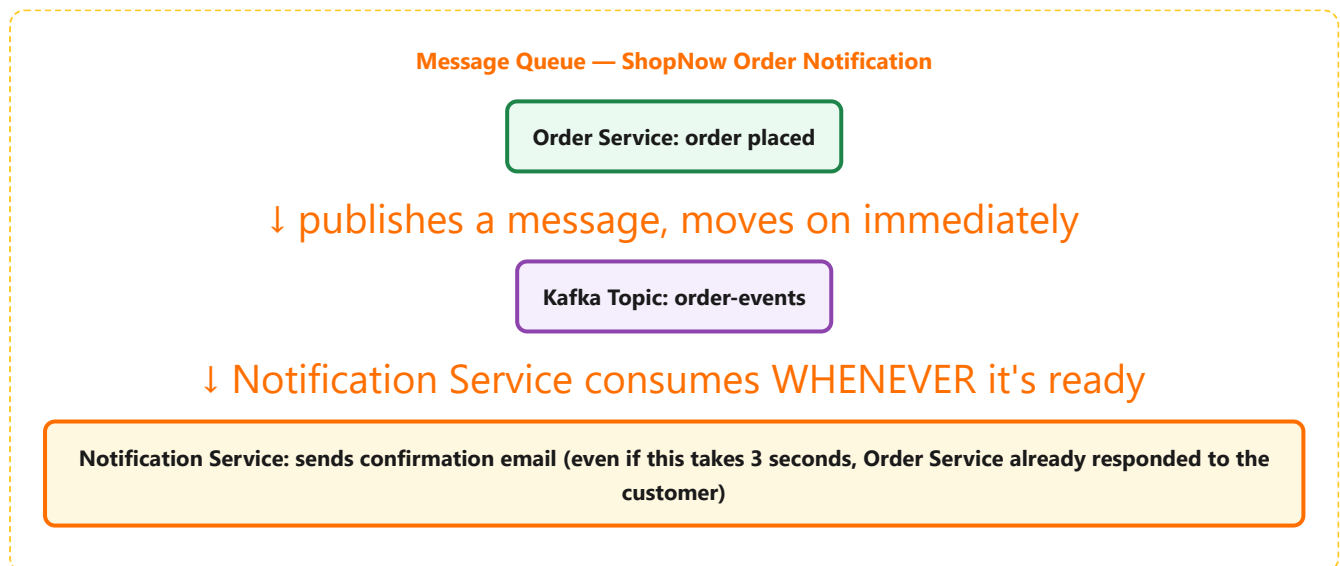
14.1 Simple Explanation

A Message Queue lets one service send a message that ANOTHER service processes later, independently — like leaving a voicemail instead of requiring the other person to answer the phone live right now.

14.2 Why It's Needed

Decouples services in time — the sender doesn't need the receiver to be available RIGHT NOW, and doesn't need to wait for it to finish processing before moving on, unlike a direct synchronous REST call.

14.3 Real Diagram



14.4 Kafka vs RabbitMQ

Aspect	Kafka	RabbitMQ
Model	Distributed commit log — durable, replayable event stream	Traditional message queue/broker (AMQP)
Throughput	Very high, built for massive event streaming	Good for moderate throughput, flexible routing
Message replay	Yes — consumers can rewind and reprocess history	No — once consumed/acked, typically gone
Best fit	Event-driven architecture, event sourcing, high-volume streams	Task queues, simpler routing needs

14.5 Java/Spring Boot Application

```
@Service
@RequiredArgsConstructor
public class OrderEventPublisher {
    private final KafkaTemplate<String, OrderPlacedEvent> kafkaTemplate;
    public void publish(OrderPlacedEvent event) {
        kafkaTemplate.send("order-events", event.getOrderId().toString(), event);
    }
}

@Component
public class NotificationListener {
    @KafkaListener(topics = "order-events", groupId = "notification-service")
    public void handle(OrderPlacedEvent event) {
        emailService.sendOrderConfirmation(event);
    }
}
```

14.6 Real-Time Scenario

Real scenario: "During a flash sale, ShopNow's email provider slowed down significantly — because Notification Service consumes from Kafka asynchronously, order PLACEMENT was completely unaffected; emails simply queued up in Kafka and were sent a few minutes later than usual once the email provider recovered, with zero impact on the actual checkout experience."

14.7 Interview Q&A

Q1. When would you choose Kafka over RabbitMQ?

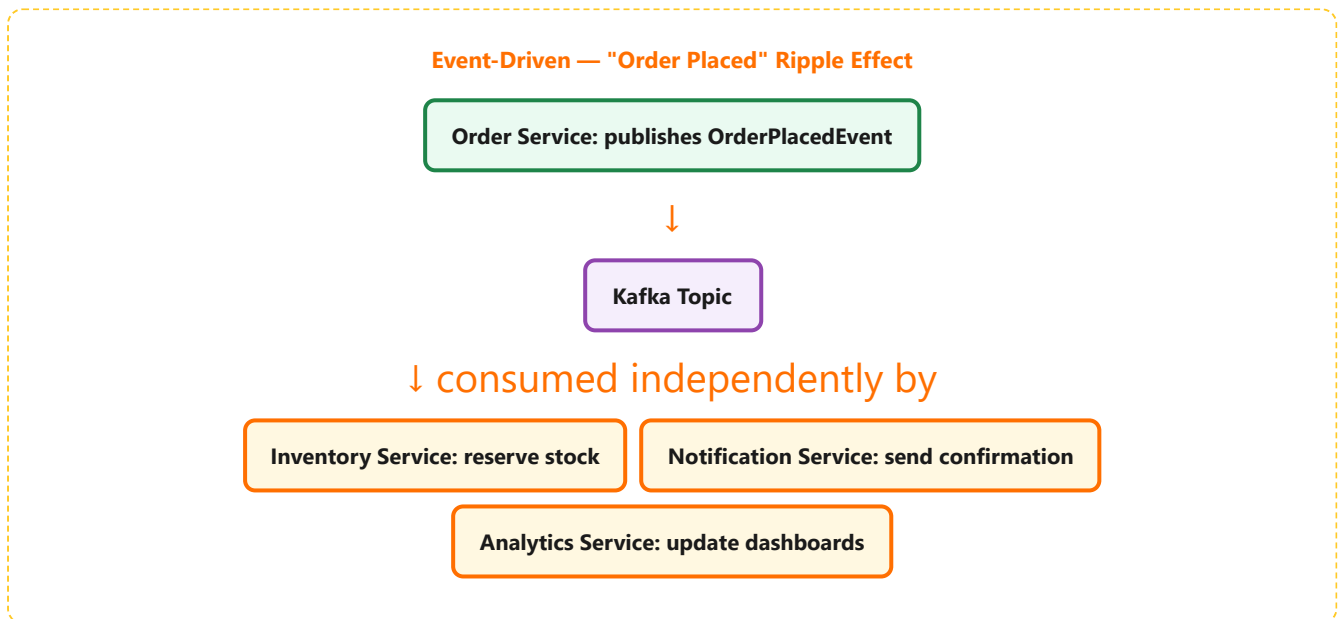
When you need very high throughput, message replay capability (reprocessing historical events), or you're building a genuinely event-driven architecture (Section 15) with multiple independent consumers of the same event stream — RabbitMQ is a better fit for simpler task-queue style workloads.

15. Event-Driven Architecture

15.1 Simple Explanation

Instead of services calling each other directly, services publish EVENTS ("something happened") and other interested services subscribe and react independently — nobody has to explicitly tell every interested party, they just listen for events they care about.

15.2 Real Diagram



15.3 Why It's Needed

Order Service doesn't need to know Inventory, Notification, or Analytics services even exist — new consumers can be added later with ZERO changes to Order Service, and if one consumer is temporarily down, the others are completely unaffected.

15.4 Real-Time Scenario

Real scenario: "When ShopNow added a new 'Loyalty Points' feature, they simply subscribed a new Loyalty Service to the EXISTING order-events topic — Order Service required zero code changes or redeployment, exactly demonstrating the decoupling this architecture is designed to provide."

15.5 Interview Q&A

Q1. What's the main trade-off of event-driven architecture compared to direct synchronous calls?

Consistency becomes eventual, not immediate — for a brief window, a downstream consumer may not yet know about an event; this is an acceptable trade-off for side effects (notifications, analytics) but requires careful design (Saga pattern, Section 16) for operations needing stronger guarantees.

16. Distributed Transactions (Saga)

16.1 Simple Explanation

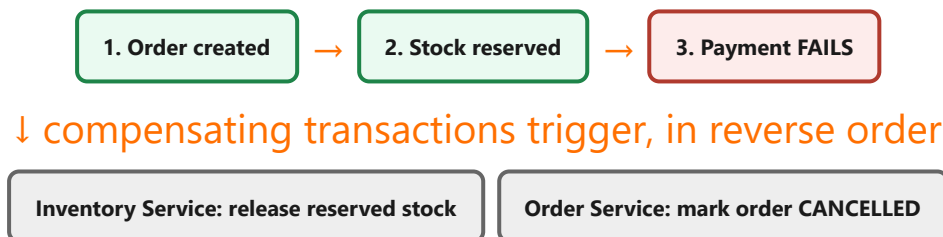
In a monolith, "create order + charge payment + reserve inventory" was ONE local database transaction. Across microservices with separate databases, there's no single transaction spanning all three — the Saga pattern replaces it with a SEQUENCE of local transactions, each with a defined "undo" (compensating) action if a later step fails.

16.2 Real Diagram

Saga — ShopNow "Place Order" Happy Path



Saga — Failure & Compensation



16.3 Choreography vs Orchestration

Style	How It Works	Best For
Choreography	Each service reacts to events independently, no central coordinator	Simple 2-3 step sagas, highly decoupled teams
Orchestration	A dedicated orchestrator service explicitly calls each step and handles compensation centrally	Complex multi-step business processes needing clear visibility of the overall flow

Full detail with real Kafka-based choreography code is already covered in the Microservices guide, Section 22-23.

16.4 Real-Time Scenario

Real scenario: "ShopNow's checkout Saga reserves inventory BEFORE charging payment — if the payment step fails (card declined), the compensating transaction releases the reserved inventory immediately, making it available to other customers

again, rather than leaving it locked indefinitely by a failed order."

16.5 Interview Q&A

Q1. Why can't you just use a traditional distributed transaction (2PC) instead of a Saga?

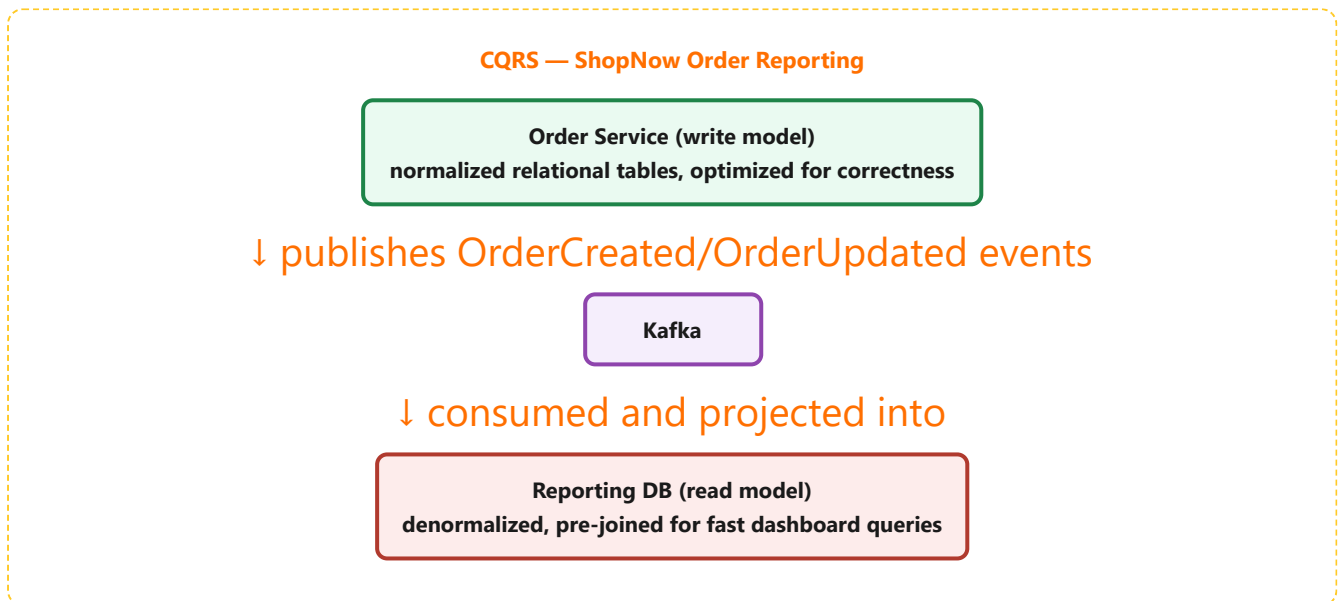
Two-Phase Commit (2PC) requires all participants to be locked/blocked until every party confirms readiness — this creates tight coupling and poor availability (if any one participant is slow/down, everyone blocks); Sagas trade strict atomicity for eventual consistency with much better availability characteristics.

17. CQRS Pattern

17.1 Simple Explanation

CQRS (Command Query Responsibility Segregation) separates the model used to WRITE data (Commands) from the model used to READ data (Queries) — they can even live in entirely different databases, each optimized independently for its own purpose.

17.2 Real Diagram



17.3 Why It's Needed

A normalized write model is great for correctness (avoiding data duplication/anomalies) but often terrible for complex reporting queries requiring many joins. CQRS lets the read side be a completely different, denormalized shape optimized purely for fast queries, without compromising the write side's correctness.

17.4 Real-Time Scenario

Real scenario: "ShopNow's 'Sales Dashboard' used to run a heavy multi-table JOIN query directly against the live Order database, slowing down actual order processing during peak reporting usage. Moving to CQRS — a separate, denormalized Reporting DB updated via Kafka events — let the dashboard query instantly without EVER touching or slowing down the live transactional Order database."

17.5 Interview Q&A

Q1. What's the real cost of adopting CQRS?

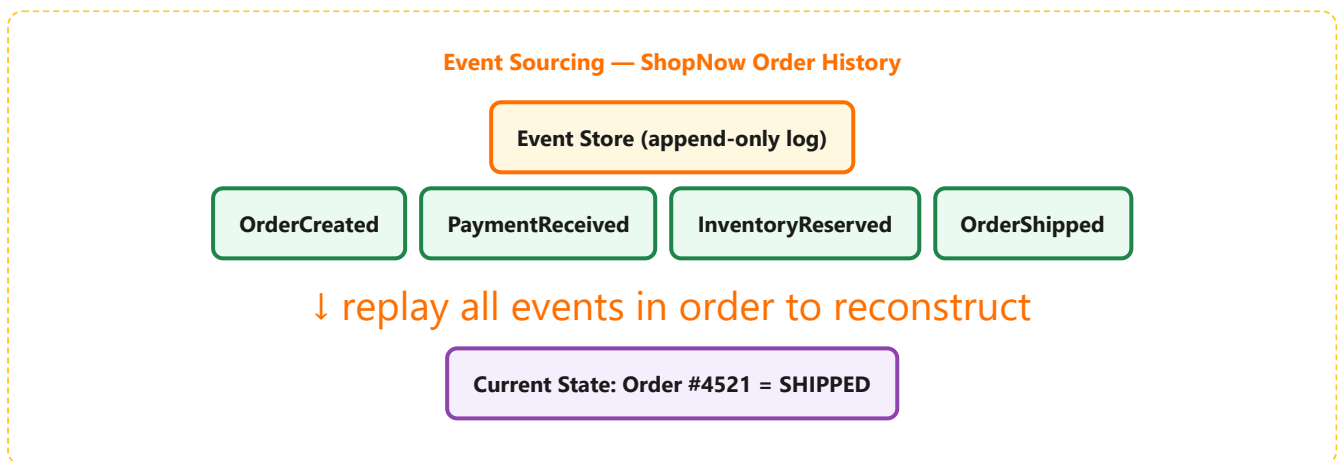
You now maintain TWO models (write and read) and an event pipeline keeping them in sync, plus the read model can be briefly stale (eventual consistency) — CQRS should only be used where the read/write access patterns genuinely differ enough to justify this complexity, not for simple CRUD services.

18. Event Sourcing

18.1 Simple Explanation

Instead of storing just the CURRENT state of an entity (e.g., "order status = SHIPPED"), Event Sourcing stores the FULL SEQUENCE of events that led to that state ("OrderCreated → PaymentReceived → InventoryReserved → OrderShipped") — the current state is derived by replaying all events, rather than being stored directly.

18.2 Real Diagram



18.3 Why It's Needed

Benefit	Real Value
Complete audit trail	Every single state change is preserved forever — exactly WHAT happened and WHEN, not just the final state
Time travel / debugging	Replay events up to any point in time to see exactly what the state looked like at that moment
Natural fit with CQRS	The event log IS the write model; multiple different read models (Section 17) can be projected from the same events

18.4 Real-Time Scenario

Real scenario: "A customer disputes their order's timeline — 'I paid three days ago, why does it show unpaid?' With Event Sourcing, ShopNow's support team can replay the EXACT sequence of events for that order, seeing precisely when each event occurred, versus a traditional system that only stores 'current status' with no history of how it got there."

18.5 Interview Q&A

Q1. What's the main challenge with Event Sourcing at scale?

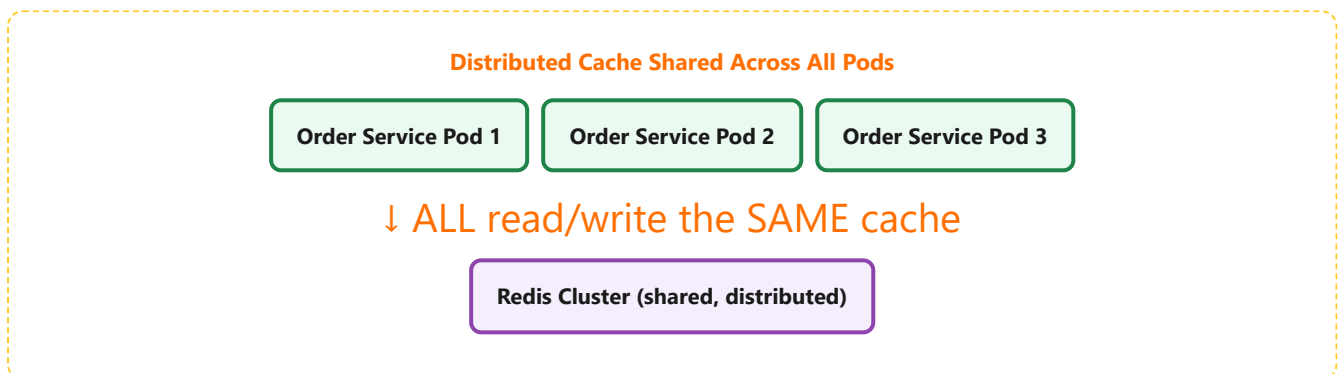
Replaying thousands of events to reconstruct current state becomes slow over time — real systems solve this with periodic "snapshots" (a saved current-state checkpoint), so replay only needs to process events since the last snapshot, not the entire history from the beginning.

19. Distributed Caching

19.1 Simple Explanation

Distributed Caching (Redis, Memcached) is a cache SHARED across all instances of a service, unlike a local in-process cache that exists separately per instance — every pod sees the SAME cached data, consistently.

19.2 Real Diagram (Recap, Applied Architecturally)



19.3 Distributed Caching at True Scale — Redis Cluster

A single Redis instance eventually hits capacity/throughput limits — Redis Cluster shards the cached data across multiple Redis nodes (exactly like database sharding, Section 5, applied to the cache layer), with each node responsible for a subset of keys, allowing the cache itself to scale horizontally.

19.4 Real-Time Scenario

Real scenario: "ShopNow's single Redis instance started showing high CPU and occasional slow responses as traffic grew — migrating to a 3-node Redis Cluster (Azure Cache for Redis Premium tier) distributed the cached data and load across all three nodes, resolving the bottleneck without any application code changes, since the Redis client library handles cluster routing transparently."

19.5 Interview Q&A

Q1. What's the difference between a local cache and a distributed cache, and when would you use each?

A local (in-process) cache is extremely fast (no network hop) but inconsistent across multiple instances. A distributed cache is shared/consistent across all instances but has a small network round-trip cost — real systems often use BOTH: a small local cache for extremely hot data, backed by a distributed cache as the shared source of truth.

20. Observability (Logs, Metrics, Traces)

20.1 Simple Explanation

Observability is having enough visibility into a running system to understand WHY something is happening, even for problems you never anticipated — built from three pillars: Logs (detailed event records), Metrics (aggregated trends), and Traces (a request's full journey across services).

20.2 Real Diagram

Using All Three Pillars — ShopNow Slow Checkout Investigation

1. Grafana metric dashboard shows: checkout p99 latency spiked



2. Distributed trace (Jaeger/App Insights) shows the Payment Service call is the slow span



3. Centralized logs (Splunk/ELK) filtered by trace ID reveal the exact error from the payment gateway

20.3 Why It's Needed in Microservices Specifically

In a monolith, a stack trace tells you everything. Across 10 microservices, a single slow customer request might touch 5 of them — without distributed tracing tying it all together via one correlation ID, finding the actual slow/failing hop becomes nearly impossible guesswork.

Full complete depth on every aspect of this topic (Splunk queries, Prometheus/Grafana, OpenTelemetry, SLA/SLO, real production troubleshooting) is covered in the dedicated Observability & Monitoring guide.

20.4 Real-Time Scenario

Real scenario: "A customer complains their checkout was slow. Support pulls the correlation ID from the customer's error screen, searches it across the observability platform, and instantly sees the full trace: Gateway (5ms) → Order Service (20ms) → Payment Service (2800ms — the bottleneck) → Inventory Service (15ms), immediately pointing at Payment Service's slow downstream card network call as the root cause."

20.5 Interview Q&A

Q1. What's the difference between monitoring and observability?

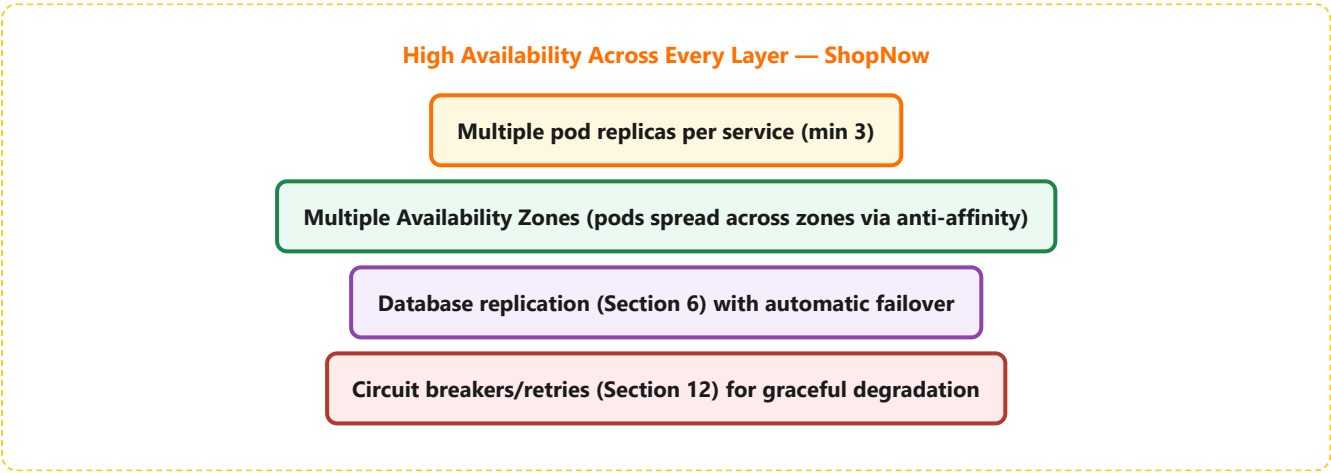
Monitoring watches predefined metrics/dashboards for KNOWN failure conditions ("is the system healthy?"). Observability provides rich enough data (logs+metrics+traces, correlated) to investigate NEW, previously unanticipated problems — monitoring is a subset of observability.

21. High Availability & Disaster Recovery

21.1 Simple Explanation

High Availability (HA) means the system keeps working correctly even when individual components fail — no single point of failure. **Disaster Recovery (DR)** is the plan for recovering the ENTIRE system after a major catastrophic event (an entire data center/region going down).

21.2 Real Diagram



21.3 HA vs DR — Key Metrics

Metric	Meaning	Real Example
RTO (Recovery Time Objective)	How long the system can be down before it's fully recovered	"We must be back online within 1 hour of a disaster"
RPO (Recovery Point Objective)	How much data loss is acceptable, measured in time	"We can lose at most 5 minutes of transactions"

21.4 Real DR Strategies

Strategy	How It Works	Trade-Off
Backup and Restore	Regular backups, restored to a new environment if disaster strikes	Cheapest, but slowest recovery (high RTO) and some data loss (RPO = time since last backup)
Pilot Light	A minimal version of the system runs constantly in a second region, scaled up only when needed	Faster recovery than backup/restore, moderate ongoing cost
Active-Passive (Warm Standby)	A full standby environment runs in a second region, ready to take over	Fast recovery (low RTO), higher ongoing cost for the idle standby capacity
Active-Active (Multi-Region)	Both regions actively serve real traffic simultaneously	Near-zero RTO/RPO, but the highest cost and complexity (data must be synchronized across regions)

21.5 Real-Time Scenario

Real scenario: "ShopNow runs an Active-Passive DR strategy — the primary region (East US) serves all live traffic, while a Warm Standby in a second region (West Europe) continuously receives database replication updates. During a real regional Azure outage, ShopNow's DNS failover redirected traffic to the standby region within 8 minutes, meeting their agreed 15-minute RTO, with less than 30 seconds of data loss thanks to near-real-time replication."

21.6 Interview Q&A

Q1. What's the difference between High Availability and Disaster Recovery?

HA handles component-level failures WITHIN a normally-operating environment (a pod crashes, a node fails) using redundancy the system automatically recovers from. DR handles catastrophic, whole-environment failures (an entire data center/region going down), requiring a deliberate failover plan to a separate environment.

Q2. How would you decide between Active-Passive and Active-Active DR for a given system?

If the business can tolerate a few minutes of downtime during a rare disaster and wants lower ongoing cost, Active-Passive is usually the right, more cost-effective choice. If the system requires near-zero downtime even during a regional disaster (e.g., a global payments platform), the higher cost and complexity of Active-Active is justified.

22. Content Delivery Network (CDN)

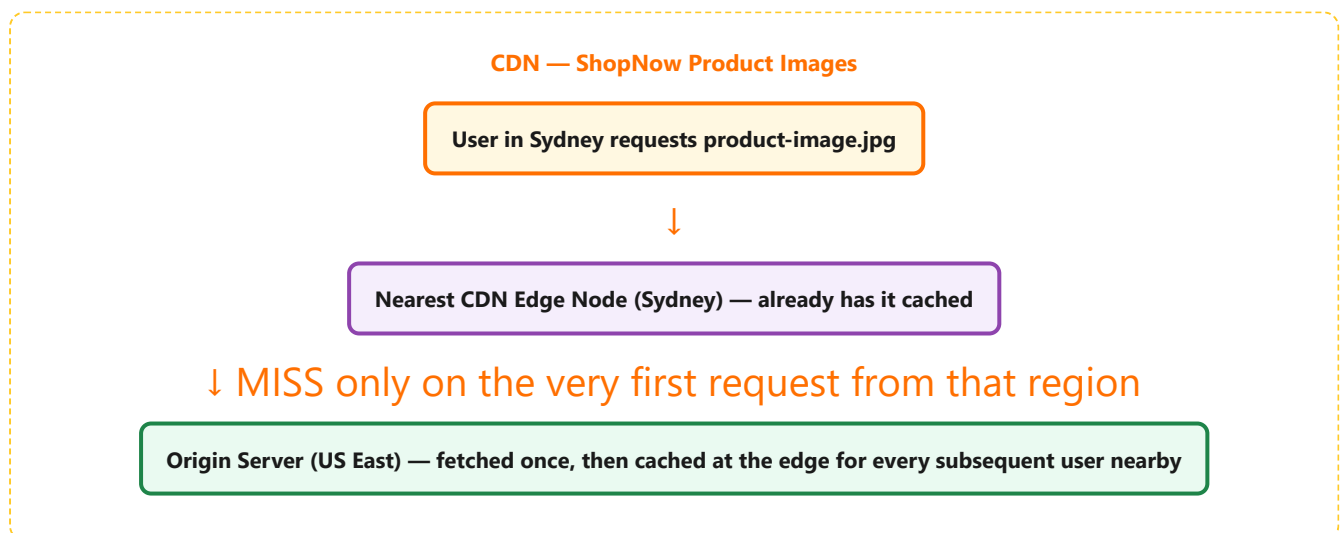
22.1 Simple Explanation

A CDN is a network of servers spread across many geographic locations that cache and serve static content (images, JS/CSS bundles, videos) from a server PHYSICALLY CLOSE to the user — instead of every user's request traveling all the way to ShopNow's origin server in one region, it's served from a nearby "edge" location.

22.2 Why It's Needed

Without a CDN, a user in Australia requesting an image hosted only on a server in the US East region suffers real network latency just from the physical distance — and the origin server must handle every single request for every single user worldwide, wasting bandwidth on identical static content repeatedly.

22.3 Real Diagram



22.4 React + Cloud Application

React's built static bundle (JS/CSS) and all product images are typically uploaded to cloud storage (Azure Blob Storage/AWS S3) and served through a CDN (Azure CDN/CloudFront/Cloudflare) rather than directly from the Spring Boot servers — keeping backend compute focused purely on dynamic API requests, while static assets are served at CDN edge speed globally.

22.5 Real-Time Scenario

Real scenario: "ShopNow's product image load time for international customers dropped from 900ms to under 50ms after moving images behind a CDN — and origin server bandwidth costs dropped significantly since the CDN absorbed the vast majority of repeat requests for the same popular product images."

22.6 Interview Q&A

Q1. What kind of content is a CDN best suited for, and what is it NOT good for?

CDNs excel at static, rarely-changing content (images, videos, JS/CSS bundles) that's the same for every user. They're a poor fit for highly personalized, frequently-changing dynamic data (like a user's live order status), which should come directly from the API instead.

Q2. How does a CDN stay updated when the underlying file changes?

Through cache invalidation/purging (explicitly telling the CDN to drop a stale file) or versioned file names (e.g., `app.a1b2c3.js`) — changing the filename on each deploy guarantees users always get the new version without needing to wait for a cache to expire.

23. Consistent Hashing

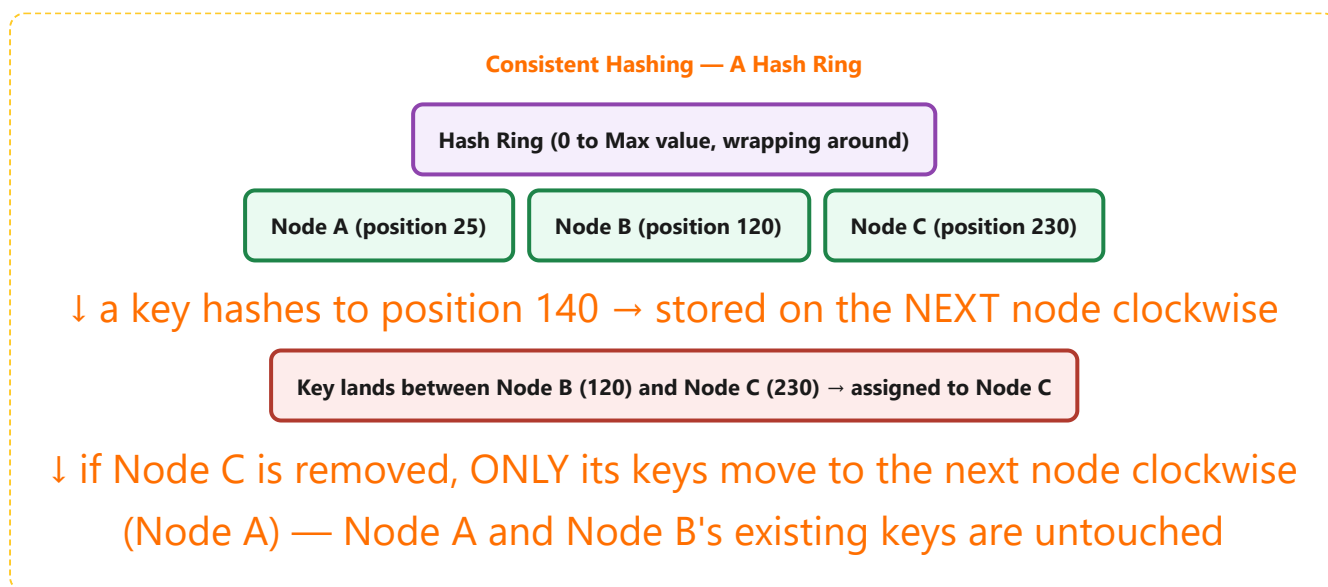
23.1 Simple Explanation

Consistent Hashing is a technique for mapping keys (like a customer ID) to servers/shards/cache nodes in a way that, when a node is ADDED or REMOVED, only a SMALL fraction of keys need to move — unlike simple modulo hashing ($\text{hash} \% N$), where changing the number of nodes N reshuffles almost EVERY key's assignment.

23.2 Why It's Needed

With simple $\text{hash}(\text{key}) \% \text{number_of_nodes}$, adding just ONE new cache/database node changes the result of the modulo for almost every existing key — meaning nearly the entire cache goes cold, or nearly all sharded data must be physically moved, all at once.

23.3 Real Diagram



23.4 Where It's Actually Used

System	How Consistent Hashing Helps
Redis Cluster (Section 19.3)	Distributes cache keys across nodes, minimizing re-distribution when scaling the cluster up/down
Database Sharding (Section 5)	An alternative to plain hash-based sharding that avoids a full re-shard when adding a new shard
Load Balancers with session affinity	Consistently routes the same client to the same backend instance even as instances scale

Distributed caches (Memcached, DynamoDB internally)	Core mechanism for partitioning keys across many nodes evenly
---	---

23.5 Real-Time Scenario

Real scenario: "When ShopNow's Redis Cluster scaled from 3 to 4 nodes during a traffic spike, consistent hashing meant only about 25% of cached keys needed to move to the new node — with simple modulo hashing, nearly ALL cached keys would have been invalidated at once, causing a massive, sudden spike in database load as everything became a cache miss simultaneously."

23.6 Interview Q&A

Q1. Why is consistent hashing better than simple modulo (hash % N) hashing when scaling a distributed cache?

Modulo hashing's result changes for almost every key whenever N (the node count) changes, causing massive data movement/cache invalidation. Consistent hashing only remaps the keys that fall between the affected node and its neighbor on the hash ring, keeping data movement proportional to the change, not the whole dataset.

24. Database Indexing

24.1 Simple Explanation

An index is an extra, separately-maintained data structure (usually a B-Tree) that lets the database find rows matching a condition WITHOUT scanning every single row — like a book's index page letting you jump straight to the right page instead of reading the entire book to find a topic.

24.2 Why It's Needed

Without an index, a query like `WHERE customer_id = 88452` on a 100-million-row table forces the database to check EVERY row (a full table scan) — as data grows, this gets progressively and dramatically slower, even though the actual matching rows might be very few.

24.3 Real Diagram

Indexed vs Non-Indexed Lookup — ShopNow Orders Table

No Index: scan all 100,000,000 rows to find `customer_id = 88452`

With Index on `customer_id`: B-Tree lookup finds matching rows in a handful of steps

24.4 Java/Spring Boot Application

```
// Real JPA entity — declaring an index explicitly
@Entity
@Table(name = "orders", indexes = {
    @Index(name = "idx_customer_id", columnList = "customer_id")
})
public class Order { ... }
```

Indexes aren't free — every INSERT/UPDATE/DELETE must also update every index on that table, so indexing every column "just in case" slows down writes. Real teams index based on ACTUAL query patterns (checked via `EXPLAIN` plans), typically on foreign keys and columns used heavily in `WHERE` / `JOIN` / `ORDER BY` clauses.

24.5 Real-Time Scenario

Real scenario: "ShopNow's 'order history' page query took 4 seconds once the Orders table crossed 50 million rows — running `EXPLAIN` revealed a full table scan on `customer_id`. Adding an index on that column brought the query down to under 10 milliseconds, with no application code change at all."

24.6 Interview Q&A

Q1. Why shouldn't you just index every column in every table?

Every additional index adds overhead to every write operation (the index must be updated too) and consumes extra storage — indexes should be added deliberately based on real, observed query patterns, not defensively on every column.

Q2. What's a composite index, and when would you use one?

An index built on MULTIPLE columns together (e.g., `customer_id, order_date`), used when queries frequently filter/sort by that exact combination — it's more effective than two separate single-column indexes for such combined queries, but column ORDER in the index matters for which query patterns it actually speeds up.

25. Idempotency & Idempotent APIs

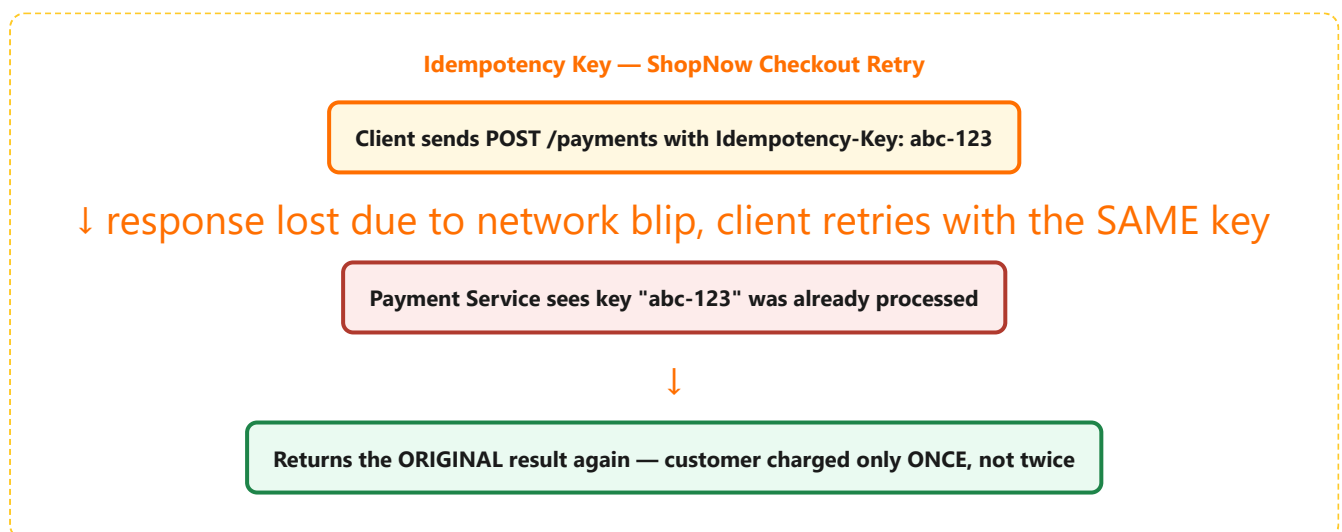
25.1 Simple Explanation

An operation is **idempotent** if performing it multiple times has the EXACT SAME effect as performing it once — like pressing an elevator call button 5 times; the elevator still only comes once, extra presses change nothing further.

25.2 Why It's Needed in Distributed Systems

Networks are unreliable — a client might send a "charge payment" request, the server processes it successfully, but the response gets lost before reaching the client. The client, seeing no response, retries the SAME request — without idempotency, this can charge the customer TWICE for one order.

25.3 Real Diagram



25.4 Java/Spring Boot Application

```
@PostMapping("/payments")
public PaymentResponse charge(@RequestHeader("Idempotency-Key") String key,
                              @RequestBody PaymentRequest request) {
    Optional<PaymentResponse> existing = idempotencyStore.get(key); // checked in Redis/DB
    if (existing.isPresent()) return existing.get(); // safe to return cached result

    PaymentResponse response = paymentProcessor.charge(request);
    idempotencyStore.save(key, response); // remember this key's result
    return response;
}
```

25.5 Which HTTP Methods Are Naturally Idempotent?

Method	Naturally Idempotent?	Why
GET	Yes	Reading data repeatedly doesn't change anything
PUT	Yes	Setting a resource to the same full state repeatedly has the same end result
DELETE	Yes	Deleting an already-deleted resource still results in it being gone
POST	No (by default)	Typically CREATES a new resource each time — needs an explicit idempotency key to be made safe for retries

25.6 Real-Time Scenario

Real scenario: "ShopNow's mobile app had unreliable network conditions in certain regions, causing checkout requests to time out client-side even though the server had already successfully processed them — the app retried automatically. Because the Payment Service enforced idempotency keys generated once per checkout attempt, these retries never resulted in a double charge, even though the exact same request hit the server 2-3 times."

25.7 Interview Q&A

Q1. Why is POST not idempotent by default, and how do you make a POST endpoint safe to retry?

A plain POST typically creates a new resource on every call — retrying it naturally creates duplicates. Making it idempotent requires the CLIENT to generate a unique idempotency key per logical operation, which the server stores and checks before processing, returning the original result on a duplicate key instead of repeating the side effect.

26. Health Checks, Readiness Probes & Blue-Green/Canary Deployments

26.1 Simple Explanation

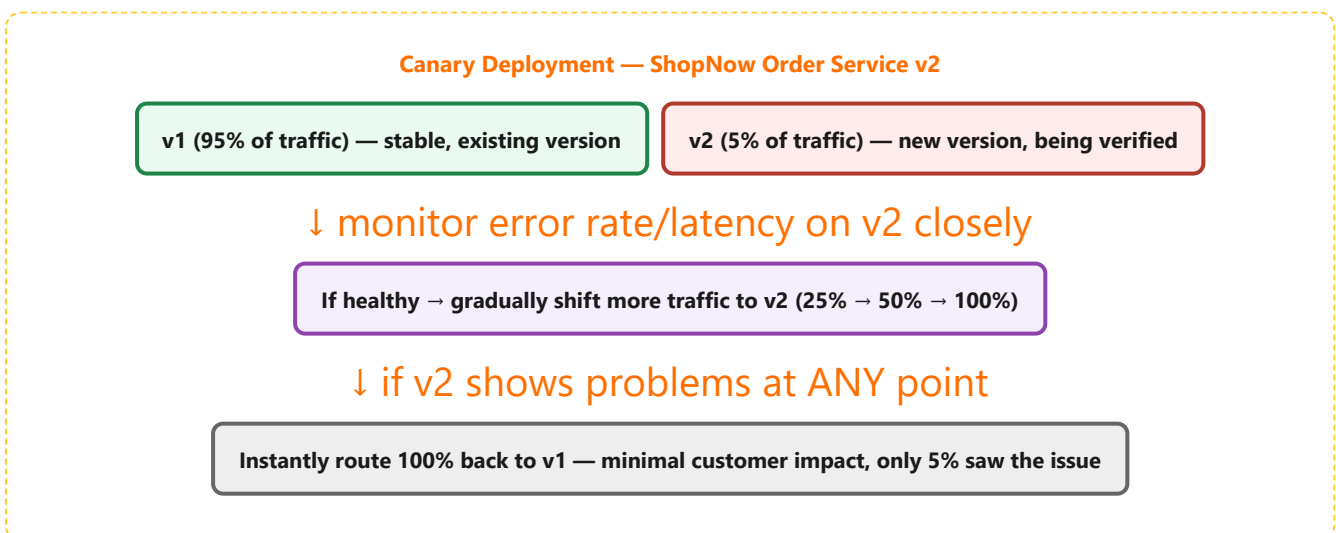
A **health check** is an endpoint (e.g., `/actuator/health`) that a Load Balancer or Kubernetes calls periodically to confirm an instance is actually working BEFORE sending it real traffic — an unhealthy instance is automatically pulled out of rotation rather than serving errors to real users.

26.2 Liveness vs Readiness Probes (Kubernetes)

Probe	Question It Answers	What Happens If It Fails
Liveness Probe	"Is this pod still alive/functioning at all?"	Kubernetes RESTARTS the pod
Readiness Probe	"Is this pod ready to receive real traffic right now?"	Kubernetes temporarily REMOVES it from the Load Balancer's rotation (no restart) until it passes again

A Spring Boot pod might be "alive" (liveness passes) but still warming up its database connection pool or loading a cache on startup — readiness correctly keeps it out of traffic during that brief window without unnecessarily restarting it.

26.3 Real Diagram — Canary Deployment



26.4 Blue-Green vs Canary vs Rolling Deployment

Strategy	How It Works	Trade-Off
Rolling Update	Old instances replaced gradually, one/few at a time	No extra infrastructure cost, but both versions run simultaneously during rollout, briefly

Blue-Green	A full parallel "Green" environment is deployed; traffic switches all at once when ready	Instant rollback (just switch back to "Blue"), but requires double the infrastructure temporarily
Canary	New version gets a small traffic percentage first, gradually increased if healthy	Safest for catching real production issues early, but requires good metrics/monitoring to judge "healthy"

26.5 Real-Time Scenario

Real scenario: "ShopNow deployed a new pricing calculation change using a Canary strategy — 5% of checkout traffic hit the new version first. Within 10 minutes, error rate metrics showed a bug affecting international currency conversion. Because only 5% of customers were exposed, ShopNow rolled back instantly with minimal real customer impact — a full Blue-Green switch to 100% would have affected every single customer instead."

26.6 Interview Q&A

Q1. Why does Kubernetes need BOTH a liveness and a readiness probe — isn't one enough?

They answer different questions — liveness decides if a pod needs restarting (it's stuck/crashed), readiness decides if it should currently receive traffic (it might be alive but temporarily not ready, e.g., during startup or a brief downstream dependency outage) — conflating them risks either unnecessary restarts or sending traffic to a not-yet-ready pod.

Q2. When would you choose Canary over Blue-Green deployment?

Canary is preferred when you want to limit the "blast radius" of a risky change by exposing it to a small percentage of real traffic first and watching real metrics before a full rollout — Blue-Green is simpler and better when you need an instant, all-or-nothing cutover with guaranteed fast rollback.

27. Back-of-the-Envelope Capacity Estimation

27.1 Simple Explanation

Before designing a system, interviewers (and real architects) often estimate rough numbers — expected users, requests per second, storage needed — using simple math, to decide whether a single server is enough or whether sharding/caching/CDNs (covered earlier) are actually necessary from day one.

27.2 A Worked Example — ShopNow Order Volume

Given: 2 million daily active users, each placing an average of 0.1 orders/day (i.e., 1 in 10 users orders something daily).

Step 1 — Total daily orders: $2,000,000 \times 0.1 = 200,000$ orders/day.

Step 2 — Average requests per second: $200,000 \div 86,400 \text{ seconds/day} \approx 2.3$ orders/second on average.

Step 3 — Peak traffic (assume 10x average during sales/peak hours): ~ 23 orders/second at peak.

Step 4 — Storage estimate: If each order record is $\sim 2\text{KB}$, $200,000 \text{ orders/day} \times 2\text{KB} \approx 400\text{MB/day} \approx 146\text{GB/year}$ of order data alone.

These rough numbers immediately tell you: 23 requests/second at peak is easily handled by a small number of horizontally-scaled pods (Section 9) behind a Load Balancer (Section 2) — you do NOT need sharding (Section 5) yet at this scale, but 146GB/year means you SHOULD plan an archival/partitioning strategy for the Orders table within a year or two.

27.3 Why This Skill Matters in Interviews

Jumping straight to "let's use Kafka and shard the database into 50 pieces" for a system that will realistically handle 5 requests/second is a common interview red flag — good system design shows you can justify EVERY architectural decision with a rough but reasoned estimate of actual scale, rather than reaching for complex patterns by default.

27.4 Interview Q&A

Q1. Why do interviewers care about back-of-the-envelope math if the numbers are just rough estimates?

The goal isn't precision — it's demonstrating that architectural decisions (sharding, caching, CDN, number of instances) are being driven by REASONED estimates of actual scale, not guesswork or over-engineering by default; a system designed for 10 requests/second looks very different from one designed for 10,000.

28. Putting It All Together — A Complete System Design Example

22.1 The Full ShopNow Architecture, Every Topic Applied Together



22.2 Why This Exercise Matters

Real system design interviews ask you to design something like ShopNow from scratch — the skill isn't memorizing these 20 topics in isolation, it's knowing WHICH ones to apply, WHERE, and WHY, for the specific system being designed. This combined diagram is exactly the kind of answer structure that demonstrates that integrated understanding.

29. Interview Questions & Real-Time Scenarios Across All Topics

Q1. Design a system that can handle 10x traffic growth without a full rewrite — what topics would you apply?

Horizontal scaling (Section 9) with load balancing (Section 2) for compute, database sharding (Section 5) and read replicas (Section 6) for data, distributed caching (Section 4/19) to reduce database load, and an API Gateway with rate limiting (Sections 3, 13) to protect against abuse at the new scale.

Q2. A checkout flow spans Order, Inventory, and Payment services — how do you ensure it either fully succeeds or fully rolls back?

The Saga pattern (Section 16) — each service performs its own local transaction with a defined compensating action, so a failure at any step triggers the appropriate rollback of previously completed steps, without needing a fragile distributed transaction.

Q3. How would you design a system to survive an entire Azure region going down?

Implement a Disaster Recovery strategy (Section 21) — at minimum Active-Passive with a warm standby in a second region, database replication (Section 6) across regions, and DNS-based failover, sized to meet the business's agreed RTO/RPO targets.

Q4. Why would you choose eventual consistency for a product review count but strong consistency for an inventory count?

A stale review count for a few seconds has near-zero real business impact (Section 8's consistency models). A stale inventory count could let a customer buy an item that's actually sold out, a real customer-facing and operational problem — the CAP Theorem (Section 7) trade-off is deliberately made differently for each based on actual business impact.

Q5. How would you prevent one slow downstream service from taking down your entire system?

Circuit Breakers with timeouts and fallbacks (Section 12) on every inter-service call, combined with Bulkheads isolating resource pools per dependency — ensuring a struggling service degrades gracefully instead of cascading failure to unrelated parts of the system.

Q6. Design a real-time notification system that must not slow down the core checkout flow.

Use an event-driven architecture (Section 15) with a message queue (Section 14) — Order Service publishes an event and immediately responds to the customer; a separate Notification Service consumes the event asynchronously, completely decoupled from checkout's response time.

Q7. How would you build a reporting dashboard without slowing down your transactional database?

Apply CQRS (Section 17) — build a separate, denormalized read model populated via events from the write side, so heavy reporting queries never touch or compete with the live transactional database.

Q8. How do independent microservices find and call each other reliably as instances scale up and down?

Service Discovery (Section 11, Eureka or Kubernetes-native DNS) — services register themselves and are looked up by name rather than hardcoded IP, so the actual set of healthy instances can change freely without breaking service-to-service calls.

Q9. A customer disputes exactly when their order status changed — how would you design the system to answer this definitively?

Event Sourcing (Section 18) — store the full sequence of events (not just current state), letting you reconstruct and prove the exact timeline of what happened and when, rather than only having the final current status available.

Q10. How would you protect your platform from a single API partner overwhelming your system?

Rate limiting (Section 13) enforced centrally at the API Gateway (Section 3), scoped per API key/partner, using a Redis-backed atomic counter — ensuring one partner's excessive traffic can't degrade the experience for every other client.

30. Common System Design Issues & Real Fixes

Issue	Cause	Real Fix
Site goes down the moment one server crashes	Single point of failure — no redundancy at the compute layer	Run multiple stateless instances behind a Load Balancer (Section 2), minimum 2-3 replicas even under normal load
Database becomes the bottleneck as traffic grows	All reads and writes hitting one database server	Add read replicas (Section 6) for read-heavy traffic, and shard (Section 5) once write volume itself outgrows a single primary
A slow downstream dependency takes the whole system down	No timeout/circuit breaker on inter-service calls	Apply Circuit Breaker + timeout (Section 12) on every synchronous call so failures degrade gracefully instead of cascading
Users see stale or inconsistent data intermittently	Eventual consistency (Section 8) chosen for data that actually needed strong consistency	Re-evaluate the CAP trade-off (Section 7) per use case — use strong consistency for anything where staleness has real business impact (inventory, balances)
One partner/client overwhelms the API for everyone else	No rate limiting enforced centrally	Enforce rate limiting at the API Gateway (Sections 3, 13), scoped per API key/client
A new feature requires touching 10 services just to add one field	Services split too finely, or boundaries drawn along the wrong lines	Revisit service boundaries (Section 10) around actual business capabilities, not arbitrary technical layers
Reporting queries slow down the live production database	Heavy analytical joins running directly against the transactional database	Apply CQRS (Section 17) with a separate, denormalized read model fed by events
An entire region outage takes the whole application offline	No Disaster Recovery plan, single-region deployment	Design at least an Active-Passive DR strategy (Section 21) with cross-region replication and DNS failover matching agreed RTO/RPO

31. Final Summary & Cheat Sheet

The 20 topics form a connected toolkit, not isolated trivia — Load Balancer + Horizontal Scaling handle compute growth; Sharding + Replication + CAP + Consistency Models handle data growth and correctness trade-offs; Microservices + Service Discovery + API Gateway + Circuit Breaker handle service decomposition and resilience; Message Queues + Event-Driven Architecture + Saga + CQRS + Event Sourcing handle asynchronous, distributed data flows; Caching + Rate Limiting protect performance and fairness; Observability + High Availability/DR ensure you can see and survive whatever goes wrong.

The interview edge: never just define a term — explain WHY it's needed (the real problem), WHERE it fits in a real architecture, and a concrete real-world trade-off, exactly the structure used throughout this guide's ShopNow example.

24.1 Quick Cheat Sheet

Topic	One-Line Meaning
Load Balancer	Distributes traffic across multiple identical instances
API Gateway	Single entry point handling routing, auth, rate limiting centrally
Caching (Redis)	Stores frequently-needed data in fast, shared memory
Database Sharding	Splits one large dataset across multiple database instances
Database Replication	Duplicates data across servers for redundancy and read scaling
CAP Theorem	Pick 2 of Consistency, Availability, Partition Tolerance during a network partition
Consistency Models	The specific guarantee a system gives about seeing the latest data
Horizontal vs Vertical Scaling	Add more machines vs make one machine bigger
Microservices	Independently deployable services, each owning a business capability
Service Discovery	Services find each other by name, not hardcoded IP
Circuit Breaker	Fails fast once a dependency's failure rate crosses a threshold
Rate Limiting	Caps how many requests a client can make in a time window
Message Queues	Decouple services in time via asynchronous messages
Event-Driven Architecture	Services react to published events instead of direct calls
Saga Pattern	Sequence of local transactions + compensations replacing a lost distributed transaction
CQRS	Separate write and read models, optimized independently
Event Sourcing	Store the full history of events, derive current state by replay
Distributed Caching	One shared cache visible consistently across all service instances
Observability	Logs + Metrics + Traces together, enough to debug the unexpected
High Availability	No single point of failure within a normally operating system
Disaster Recovery	The plan for recovering from a catastrophic, whole-environment failure

CDN	Geographically distributed edge servers caching static content close to users
Consistent Hashing	Maps keys to nodes so scaling up/down only moves a small fraction of data
Database Indexing	A B-Tree structure letting queries find rows without scanning the whole table
Idempotency	Repeating the same operation/request has the same effect as doing it once
Health Checks / Probes	Automated checks ensuring only healthy instances receive real traffic
Canary / Blue-Green Deployment	Strategies to roll out new versions safely with limited blast radius
Capacity Estimation	Rough math on expected traffic/storage to justify architectural decisions

Use this guide alongside the [Microservices guide](#), the [Docker & Kubernetes guide](#), the [JDBC/Hibernate/Spring Data JPA guide](#), the [SQL/Spring Batch guide](#), the [Observability & Monitoring guide](#), and the [AWS/Azure Cloud Architecture guide](#) — together they connect these system design fundamentals to their complete, production-grade real-world implementation detail.